

CyberAudit

ReportViewerPage

PDF Report Viewer : Edit Mode · Read Mode · Security Score · PDF Download

Project / Projet	CyberAudit Platform
File	src/pages/ReportViewerPage.jsx
Route	/admin/report/:reference
Modes	Edit mode (isAdmin && status != completed) / Read mode
Score formula	100 - sum(SEV_WEIGHTS per finding), clamped [0,100]
Grade thresholds	>=90=A, >=80=B+, >=70=B, >=55=C, >=40=D, >=20=E, else F
PDF generation	POST /audits/:id/generate-report/ → WeasyPrint (server-side)
PDF download	GET /audits/:id/report/ as blob (202=in progress, 4xx=unavailable)
XSS protection	DOMPurify.sanitize() with ALLOWED_TAGS:[] ALLOWED_ATTR:[]
Date	June 2026 / Juin 2026

1) Page Overview & Dual Mode

1) Vue d'ensemble et double mode

ReportViewerPage serves two audiences in the same component. For an admin viewing a non-completed audit, it shows an interactive editor (Edit mode) to input findings, summary, and verdict, then generate the PDF. For a completed audit, or any non-admin viewer, it shows the rendered report in Read mode with score, grade, findings table, and a PDF download button.

ReportViewerPage sert deux audiences dans le même composant. Pour un admin consultant un audit non termine, il affiche un éditeur interactif (mode Edition) pour saisir les findings, le résumé et le verdict, puis générer le PDF. Pour un audit termine, où tout visiteur non-admin, il affiche le rapport rendu en mode Lecture avec le score, la note, le tableau des findings et un bouton de téléchargement PDF.

Mode décision tree / Arbre de décision du mode

```
// Mode set on data load:
setEditMode(isAdmin && aud?.status !== 'completed');
//
// isAdmin=true + status='pending'/'in_progress' → Edit mode
// isAdmin=true + status='completed' → Read mode
// isAdmin=false + any status → Read mode
//
// Admin can manually switch back from Read → Edit:
// <button onClick={() => setEditMode(true)}>Edit report</button>
```

Data fetch : parallel Promise.all

```
// Load both report and audit objects in parallel
const [rep, aud] = await Promise.all([
  getReportByReference(reference), // GET /reports/?reference=CA-...
  getRequestByReference(reference), // GET /audits/requests/?reference=CA-...
]);
setReport(rep);
setAudit(aud);
setSummary(rep?.summary || '');
setVerdict(rep?.verdict || '');
setFindings(Array.isArray(rep?.findings) ? rep.findings : []);
setEditMode(isAdmin && aud?.status !== 'completed');
```

Not-found guard / Garde not-found

```
// Show 'no report' state only when:
// - loading is done
// - report is null (API returned nothing)
// - AND we are not in edit mode (new report being created)
if (!report && !editMode) return <NoReportCard />;
// If editMode=true and report=null → admin creating first report → OK
// If editMode=false and report=null → nothing to show → show not-found
```

2) State Variables

2) Variables d'état

State var	Initial	Purpose EN / FR
report	null	Full Report object from API (or null). Contains grade, security_score, verdict, summary, findings[]. Read mode renders from this. Objet Report complet de l'API (ou null). Contient grade, security_score, verdict, summary, findings[]. Le mode Lecture se rend depuis ceci.
audit	null	Full AuditRequest object. Used for audit.id (PDF download), audit.status (mode decision), audit.reference. Objet AuditRequest complet. Utilise pour audit.id (telechargement PDF), audit.status (decision mode), audit.reference.
loading	true	true while Promise.all fetch is in flight. Shows spinner, hides page content. true pendant le chargement Promise.all. Affiche le spinner, cache le contenu de la page.
notice	"	Inline amber banner message. Shown on success (score/grade result), errors, and PDF status (202/4xx). Message de bannière ambre en ligne. Affiche en succes (resultat score/grade), erreurs, et statut PDF (202/4xx).
downloading	false	true while PDF blob download request is in flight. Disables the Download PDF button. true pendant la requête de téléchargement blob PDF. Désactive le bouton Telecharger PDF.
editMode	false	Drives which JSX branch renders. Set by mode decision on load. Can be toggled to true by admin. Pilote quelle branche JSX se rend. Defini par la décision de mode au chargement. Peut être bascule a true par l'admin.
findings	[]	Array of finding objects in edit mode. Populated from rep.findings on load. Mutated by addFinding/removeFinding. Tableau d'objets finding en mode édition. Peuple depuis rep.findings au chargement. Mute par addFinding/removeFinding.

summary	"	Textarea value for executive summary. Sent to generateReportFromFindings(). Pre-filled from rep.summary. Valeur textarea pour le resume executif. Envoyé a generateReportFromFindings(). Pre-rempli depuis rep.summary.
verdict	"	Input value for one-line risk verdict (maxLength 255). Sent to generateReportFromFindings(). Pre-filled from rep.verdict. Valeur input pour le verdict de risque mono-ligne (maxLength 255). Envoyé a generateReportFromFindings().
draft	EMPTY_FINDING	Controlled form state for the 'Add vulnerability' form. Reset to EMPTY_FINDING after addFinding(). Etat de formulaire controle pour le formulaire 'Ajouter une vulnerabilite'. Remis a EMPTY_FINDING apres addFinding().
submitting	false	true while handleGenerate() POST is in flight. Shows the loading modal overlay + cycling messages. true pendant le POST handleGenerate(). Affiche le modal de chargement en superposition + messages cycliques.
progressMsg	"	Current message displayed in the loading modal. Cycled by setInterval every 600ms while submitting=true. Message actuel affiché dans le modal de chargement. Cycle par setInterval toutes les 600ms pendant submitting=true.

3) Helper Functions

3) Fonctions utilitaires

severityClasses() : Tailwind badge colour

```
function severityClasses(severity) {  
  const s = (severity || '').toLowerCase();  
  if (s === 'critical') return 'bg-red-200 text-red-900';  
  if (s === 'high')     return 'bg-red-100 text-red-700';  
  if (s === 'medium')   return 'bg-amber-100 text-amber-700';  
  return 'bg-green-100 text-green-700'; // low or unknown  
}  
// Applied to <span> inside findings table (both edit and read mode)
```

scoreColor() : progress bar fill colour

```
function scoreColor(score) {  
  if (score >= 70) return 'bg-green-500'; // good  
  if (score >= 40) return 'bg-orange-500'; // warning  
  return 'bg-red-500';                     // critical  
}  
// Applied to the inner <div> of the score progress bar  
// Used for both client-side previewScore and server-side report.security_score
```

gradeColor() : grade badge colour

```
function gradeColor(grade) {  
  if (['A', 'B+'].includes(grade)) return 'bg-green-100 text-green-700';  
  if (['B', 'C'].includes(grade))  return 'bg-amber-100 text-amber-700';  
  return 'bg-red-100 text-red-700'; // D, E, F  
}  
// Applied to the 56x56px grade badge square (h-14 w-14 rounded-lg)
```

previewScore() + previewGrade() : live client-side preview

```
const SEV_WEIGHTS = { critical: 25, high: 10, medium: 4, low: 1 };  
const GRADE_TH =  
  [[90, 'A'], [80, 'B+'], [70, 'B'], [55, 'C'], [40, 'D'], [20, 'E'], [0, 'F']];  
  
function previewScore(findings) {  
  if (!findings?.length) return 100; // no findings = perfect score  
  const d = findings.reduce(  
    (s, f) => s + (SEV_WEIGHTS[(f.severity || '').toLowerCase()] ?? 1), 0  
  );  
  return Math.max(0, Math.min(100, 100 - d));  
}  
  
function previewGrade(score) {  
  for (const [t, g] of GRADE_TH) if (score >= t) return g;  
  return 'F';  
}
```

```
}  
// INDICATIVE ONLY – server recomputes the authoritative score on generate
```

sanitize() : DOMPurify XSS defense

```
function sanitize(text) {  
  return DOMPurify.sanitize(text || '', {  
    ALLOWED_TAGS: [], // strip ALL HTML tags  
    ALLOWED_ATTR: [], // strip ALL attributes  
  });  
}  
// Applied to: report.verdict, report.summary in read mode  
// Whitespace-pre-line on summary paragraph preserves line breaks  
// NOT applied in edit mode inputs (user is editing their own content)
```

getCurrentUser() : localStorage session read

```
function getCurrentUser() {  
  try {  
    const raw = localStorage.getItem('cyberaudit:session');  
    return raw ? JSON.parse(raw).user : null;  
  } catch {  
    return null; // parse error = treat as unauthenticated  
  }  
}  
const currentUser = getCurrentUser();  
const isAdmin = currentUser?.role === 'admin';  
// Note: uses localStorage directly, not useAuth() hook  
// This avoids prop drilling and works outside React context
```

4) Loading Modal : Cycling Progress Messages

4) Modal de chargement : Messages de progression cycliques

During PDF generation (`submitting=true`), a fullscreen overlay modal blocks the UI and shows a spinning indicator with rotating status messages. The messages cycle every 600ms via `setInterval`. The first message fires immediately via `tick()` before the interval starts. The interval is cleaned up via the `useEffect` cleanup function.

Pendant la generation PDF (`submitting=true`), un modal en superposition plein ecran bloque l'interface et affiche un indicateur de rotation avec des messages de statut cycliques. Les messages changent toutes les 600ms via `setInterval`. Le premier message se déclenche immédiatement via `tick()` avant le debut de l'intervalle. L'intervalle est nettoyé via la fonction de nettoyage `useEffect`.

```
// useEffect triggered when submitting changes
useEffect(() => {
  if (!submitting) return; // no-op when not generating
  const messages = [
    'Computing security score...',
    'Saving vulnerabilities...',
    'Rendering HTML report...',
    'Generating PDF with WeasyPrint...',
    'Finalizing...',
  ];
  let i = 0;
  const tick = () => {
    setProgressMsg(messages[i]);
    i = (i + 1) % messages.length; // wraps at end
  };
  const interval = setInterval(tick, 600);
  tick(); // show first message immediately (no 600ms wait)
  return () => clearInterval(interval); // cleanup on unmount or submitting=false
}, [submitting]);
```

Modal JSX : overlay + card

```
// Fixed fullscreen overlay (z-50, covers entire viewport)
{submitting && (
  <div className='fixed inset-0 z-50 flex items-center justify-center
    bg-black/60 backdrop-blur-sm'>
    <div className='mx-4 max-w-md rounded-xl bg-white p-8 shadow-2xl'>
      <div className='flex flex-col items-center gap-4'>
        {/* 56px spinner */}
        <div className='h-14 w-14 animate-spin rounded-full
          border-4 border-gray-200 border-t-brand' />
        <div className='text-center'>
          <h3 className='text-lg font-bold text-brand'>
            Report generation in progress
          </h3>
          <p className='mt-2 text-sm text-gray-500'>
            {progressMsg || 'Initializing...'}
          </p>
          <p className='mt-3 text-xs text-gray-400'>
            This takes about 3 seconds.
          </p>
        </div>
      </div>
    </div>
  )}
```

```
        </p>
      </div>
    </div>
  </div>
</div>
  ) }
```

Message 1	'Computing security score...' : shown immediately on tick()
Message 2	'Saving vulnerabilities...' : after 600ms
Message 3	'Rendering HTML report...' : after 1200ms
Message 4	'Generating PDF with WeasyPrint...' : after 1800ms
Message 5	'Finalizing...' : after 2400ms, then loops

5) Edit Mode : Findings Editor

5) Mode Edition : Editeur de findings

Edit mode renders four cards in sequence: (1) live preview score, (2) verdict + summary inputs, (3) add-vulnerability form, (4) findings list. A Generate button at the bottom triggers `handleGenerate()`. The page header shows an amber 'Edit mode' badge next to the title.

Le mode édition rend quatre cartes en séquence : (1) aperçu score live, (2) inputs verdict + résumé, (3) formulaire d'ajout de vulnérabilité, (4) liste des findings. Un bouton Generate en bas déclenche `handleGenerate()`. L'en-tête de page affiche un badge ambre 'Edit mode' à cote du titre.

Card 1 : Live preview score

```
// Live score computed from current findings array
const pScore = previewScore(findings);
const pGrade = previewGrade(pScore);

<div className='rounded-xl border border-gray-100 bg-white p-5 shadow-sm'>
  <h2>Preview score (live)</h2>
  { /* 56x56 grade badge */ }
  <span className={ `flex h-14 w-14 items-center justify-center rounded-lg
    text-2xl font-extrabold ${gradeColor(pGrade)} ` }>
    {pGrade}
  </span>
  { /* Score bar - width = pScore% */ }
  <div className='h-3 w-full overflow-hidden rounded-full bg-gray-200'>
    <div className={ `h-full rounded-full ${scoreColor(pScore)} ` }
      style={{ width: `${pScore}%` }} />
  </div>
  <p>{pScore} / 100 - recalculated server-side on generation</p>
</div>
```

Card 2 : Verdict & Summary inputs

verdict <input>	text, maxLength=255. One-line risk level summary (e.g. 'Overall risk level: Concerning'). Sent to <code>generateReportFromFindings()</code>. Texte, maxLength=255. Resume mono-ligne du niveau de risque. Envoye a <code>generateReportFromFindings()</code>.
summary <textarea>	rows=5. Full executive summary of scope, methodology, main findings. Sent to <code>generateReportFromFindings()</code>. rows=5. Resume executif complet de la portee, methodologie, principaux findings. Envoye a <code>generateReportFromFindings()</code>.

Card 3 : Add vulnerability form

```
// draft state = controlled finding form
const EMPTY_FINDING = { severity: 'Medium', asset: '', description: '',
  recommendation: '' };
const [draft, setDraft] = useState({ ...EMPTY_FINDING });

function addFinding() {
  if (!draft.asset.trim()) {
    setNotice("The 'Asset' field is required."); // only asset is required
    return;
  }
  setFindings([...findings, { ...draft }]); // append to array
  setDraft({ ...EMPTY_FINDING });          // reset form
  setNotice('');                            // clear error
}

// Form grid: 2 cols on sm:
// Col 1: Severity <select> (Critical/High/Medium/Low)
// Col 2: Asset <input> text, maxLength=200 — REQUIRED
// Row 2 sm:col-span-2: Description <textarea> rows=2
// Row 3 sm:col-span-2: Recommendation <textarea> rows=2
```

Card 4 : Findings list (edit mode)

The findings table in edit mode has 5 columns: Severity badge, Asset, Description, Recommendation, and a Remove button. Each row has a `removeFinding(i)` button that filters the finding out of the array by index. The table is horizontally scrollable (`overflow-x-auto`) with `min-w-[640px]`.

Le tableau de findings en mode édition a 5 colonnes : badge Severity, Asset, Description, Recommendation, et un bouton Remove. Chaque ligne a un bouton `removeFinding(i)` qui filtre le finding du tableau par index. Le tableau est horizontalement scrollable (`overflow-x-auto`) avec `min-w-[640px]`.

```
function removeFinding(i) {
  setFindings(findings.filter((_, idx) => idx !== i));
}

// Table columns (edit mode only):
// Severity | Asset | Description | Recommendation | — (remove button)

// Severity badge:
<span className={`rounded-full px-2 py-0.5 text-xs font-semibold
  ${severityClasses(f.severity)}`}>
  {f.severity}
</span>
```

6) handleGenerate() : PDF Generation Flow

6) handleGenerate() : Flux de generation PDF

handleGenerate() is the main submit action in edit mode. It POSTs findings+summary+verdict to the Django backend, which computes the authoritative score, renders HTML with Jinja2, and converts to PDF with WeasyPrint. On success it auto-marks the audit as completed, shows the score notice, then reloads both report and audit before switching to read mode.

handleGenerate() est l'action de soumission principale en mode edition. Ce POSTs findings+summary+verdict au backend Django, qui calcule le score officiel, rend le HTML avec Jinja2, et convertit en PDF avec WeasyPrint. En succes il marque automatiquement l'audit comme complète, affiche la notice de score, puis recharge le rapport et l'audit avant de basculer en mode lecture.

```
async function handleGenerate() {
  if (!audit?.id) return; // guard: no audit loaded
  setSubmitting(true);    // shows modal loader
  setNotice('');
  try {
    // POST findings + summary + verdict to backend
    const result = await generateReportFromFindings(reference, {
      summary, verdict, findings,
    });
    // result = { security_score, grade, findings_count, ... }

    // Auto-mark as completed if not already
    if (audit.status !== 'completed') {
      await updateRequest(reference, { status: 'completed' });
    }

    // Show score notice for 5 seconds
    setNotice(
      `Report generated (score ${result.security_score}/100, grade  

      ${result.grade}). PDF in progress...`
    );
    setTimeout(() => setNotice(''), 5000);

    // Reload both objects in parallel
    const [fresh, freshAudit] = await Promise.all([
      getReportByReference(reference),
      getRequestByReference(reference),
    ]);
    setReport(fresh);
    setAudit(freshAudit);
    setEditMode(false); // switch to read mode
  } catch (e) {
    setNotice('Error during generation: ' + (e.response?.data?.detail ||
    e.message));
  } finally {
    setSubmitting(false); // hides modal loader
  }
}
```

Generate button label / Libelle du bouton Generate

```
// Button label shows findings count and changes during submission
<button onClick={handleGenerate} disabled={submitting}>
  {submitting
    ? 'Generating...'
    : `Generate report & mark Completed (${findings.length} findings)`}
</button>
// disabled:opacity-50 - button greyed out while submitting
```

7) Read Mode : Completed Report Display

7) Mode Lecture : Affichage du rapport termine

Read mode renders three cards: (1) global security score with grade badge and score bar, (2) executive summary, (3) vulnerabilities table. All text fields are sanitized with DOMPurify. An admin-only 'Edit report' button allows switching back to edit mode to revise and regenerate.

Le mode lecture rend trois cartes : (1) score de securite globale avec badge de note et barre de score, (2) resume executif, (3) tableau des vulnerabilites. Tous les champs texte sont assainis avec DOMPurify. Un bouton 'Edit report' reserve a l'admin permet de revenir en mode édition pour réviser et régénérer.

Card 1 : Global security score

```
// report.grade and report.security_score come from the server
<div className='rounded-xl border border-gray-100 bg-white p-5 shadow-sm'>
  <h2>Global security score</h2>
  { /* Grade badge - 56x56 square */ }
  <span className={`flex h-14 w-14 items-center justify-center rounded-lg
    text-2xl font-extrabold ${gradeColor(report.grade)}`} >
    {report.grade}
  </span>
  { /* Verdict line - sanitized */ }
  <p>{sanitize(report.verdict) || 'Audit completed'}</p>
  { /* Score bar */ }
  <div className='h-3 w-full overflow-hidden rounded-full bg-gray-200'>
    <div className={`h-full rounded-full ${scoreColor(report.security_score)} `}
      style={{ width: `${report.security_score}%` }} />
  </div>
  <p>{report.security_score} / 100</p>
</div>
```

Card 2 : Executive summary

```
<div className='rounded-xl border border-gray-100 bg-white p-5 shadow-sm'>
  <h2>Executive summary</h2>
  { /* whitespace-pre-line: preserves line breaks from textarea input */ }
  { /* sanitize(): strips all HTML tags via DOMPurify */ }
  <p className='whitespace-pre-line text-sm text-gray-600'>
    {sanitize(report.summary) || '-'}
  </p>
</div>
```

Card 3 : Findings table (read mode)

The read mode findings table has 4 columns (no Remove button): Severity badge, Asset, Description, Recommendation. The same severityClasses() helper applies coloured badges. The findings come from report.findings (server-returned), not the local findings state variable.

Le tableau de findings en mode lecture a 4 colonnes (pas de bouton Remove) : badge Severity, Asset, Description, Recommendation. Le meme helper severityClasses() applique les badges colores. Les findings viennent de report.findings (retournes par le serveur), pas de la variable d'état findings locale.

Read mode : colour quick reference / Référence rapide des couleurs

Grade	Score range	Grade badge	Score bar colour EN / FR
A	>= 90	bg-green-100 text-green-700	bg-green-500 (score >= 70) bg-green-500 (score >= 70)
B+	>= 80	bg-green-100 text-green-700	bg-green-500 (score >= 70) bg-green-500 (score >= 70)
B	>= 70	bg-amber-100 text-amber-700	bg-green-500 (score >= 70) bg-green-500 (score >= 70)
C	>= 55	bg-amber-100 text-amber-700	bg-orange-500 (score >= 40) bg-orange-500 (score >= 40)
D	>= 40	bg-red-100 text-red-700	bg-orange-500 (score >= 40) bg-orange-500 (score >= 40)
E	>= 20	bg-red-100 text-red-700	bg-red-500 (score < 40) bg-red-500 (score < 40)
F	0-19	bg-red-100 text-red-700	bg-red-500 (score < 40) bg-red-500 (score < 40)

8) handleDownloadPdf() : Blob Download

8) handleDownloadPdf() : Téléchargement blob

handleDownloadPdf() fetches the PDF as a binary blob from GET /audits/:id/report/. The response status drives three possible outcomes: 202 (PDF still being generated), 4xx (no PDF available), or 200 (download triggered via programmatic anchor click). revokeObjectURL cleans up the temporary blob URL immediately after the download starts.

handleDownloadPdf() récupère le PDF comme un blob binaire depuis GET /audits/:id/report/. Le statut de réponse pilote trois résultats possibles : 202 (PDF encore en cours de génération), 4xx (pas de PDF disponible), ou 200 (téléchargement déclenche via un clic d'ancre programmatique). revokeObjectURL nettoie l'URL blob temporaire immédiatement après le début du téléchargement.

```
async function handleDownloadPdf() {
  if (!audit?.id) { setNotice('Audit ID not found.');
```

`return; }
 setDownloading(true);
 setNotice('');
 try {
 const response = await api.get(`/audits/${audit.id}/report/`, {
 responseType: 'blob', // receive binary data
 validateStatus: (s) => s < 500, // don't throw on 202/4xx
 });
 if (response.status === 202) {
 setNotice('PDF being generated. Retry in a few seconds.');``return;`
 }
 if (response.status >= 400) {
 setNotice('No PDF available.');`return;`
 }
 // HTTP 200 – trigger browser download
 const url = URL.createObjectURL(
 new Blob([response.data], { type: 'application/pdf' })
);
 const a = document.createElement('a');
 a.href = url;
 a.download = `rapport-\${reference}.pdf`; // filename: rapport-CA-2024-0042.pdf
 document.body.appendChild(a);
 a.click();
 document.body.removeChild(a);
 URL.revokeObjectURL(url); // release memory
 } catch {
 setNotice('Error during download.');
 } finally {
 setDownloading(false);
 }
}

PDF download status handling / Gestion des statuts de téléchargement PDF

HTTP status	Meaning	UI behaviour EN / FR
202	PDF still generating (WeasyPrint running)	setNotice('PDF being generated. Retry in a few seconds.') — button re-enables after finally. setNotice('PDF being generated...') — bouton re-active apres finally.
200	PDF ready, binary blob returned	Blob URL created, programmatic <a> click triggers browser Save dialog. Filename: rapport-{reference}.pdf. URL blob crée, clic <a> programmatique déclenche la boîte de dialogue Enregistrer du navigateur.
>= 400	No PDF available or not found	setNotice('No PDF available.') : no file downloaded. setNotice('No PDF available.') : aucun fichier téléchargé.
catch	Network error or server 500+	setNotice('Error during download.') setNotice('Error during download.')

9) Security Scoring System

9) Systeme de scoring de security

The security score is computed in two places: client-side for a live preview while editing (previewScore), and server-side on report generation (the authoritative value stored in report.security_score). Both use the same SEV_WEIGHTS deduction formula. The grade is derived from GRADE_TH thresholds.

Le score de securite est calculé en deux endroits : cote client pour un aperçu live pendant l'édition (previewScore), et cote serveur lors de la génération du rapport (la valeur officielle stockée dans report.security_score). Les deux utilisent la même formule de déduction SEV_WEIGHTS. La note est dérivée des seuils GRADE_TH.

SEV_WEIGHTS déduction table / Tableau de déduction SEV_WEIGHTS

Critical	High	Medium	Low
-25 points	-10 points	-4 points	-1 point
bg-red-200 text-red-900	bg-red-100 text-red-700	bg-amber-100 text-amber-700	bg-green-100 text-green-700

GRADE_TH thresholds / Seuils GRADE_TH

A	B+	B	C	D	E	F	Score color
>=90	>=80	>=70	>=55	>=40	>=20	0+	>=70: green-500 >=40: orange-500 <40: red-500

Score calculation examples / Exemples de calcul

Findings	Deduction	Score	Grade
None (empty array)	0	100	A
1 Medium	-4	96	A
1 High + 2 Medium	-10 -4 -4 = -18	82	B+
1 Critical + 1 High	-25 -10 = -35	65	B
2 Critical + 3 High	-50 -30 = -80	20	E
4 Critical + 2 High	-100 -20 = -120 → clamped	0	F

10) Summary

10) Résumé

Concept	Key detail EN / FR
Dual mode	Edit (isAdmin && status!=completed) or Read. Set on load, toggle via admin button. Edition (isAdmin && status!=completed) ou Lecture. Défini au chargement, basculable via bouton admin.
Parallel load	Promise.all([getReportByReference, getRequestByReference]) : one round-trip for both objects. Promise.all([...]) : un aller-retour pour les deux objets.
Loading modal	Fixed z-50 overlay. 5 messages cycling every 600ms via setInterval, first fires immediately via tick(). Superposition fixe z-50. 5 messages cycliques toutes les 600ms via setInterval, le premier se déclenche immédiatement.
previewScore	Client-side live preview only. sum(SEV_WEIGHTS) deducted from 100, clamped [0,100]. Indicative. Aperçu live cote client uniquement. sum(SEV_WEIGHTS) déduit de 100, borne [0,100]. Indicatif.
handleGenerate	POST findings+summary+verdict. Server computes score, renders HTML, WeasyPrint→PDF. Auto-completes audit. Reloads → read mode. POST findings+summary+verdict. Serveur calcule score, rend HTML, WeasyPrint→PDF. Marque audit complete. Recharge → mode lecture.
addFinding guard	Only asset field is required. Description and recommendation are optional free-text. Seul le champ asset est requis. Description et recommandation sont des textes libres optionnels.
DOMPurify	Read mode only: report.verdict + report.summary sanitized with ALLOWED_TAGS:[] ALLOWED_ATTR:[]. Strips all HTML. Mode lecture uniquement : report.verdict + report.summary assainis avec ALLOWED_TAGS:[] ALLOWED_ATTR:[]. Supprime tout HTML.
whitespace-pre-line	Applied to summary paragraph in read mode. Preserves newlines from the admin's textarea input. Applique au paragraphe résumé en mode lecture. Préserve les retours à la ligne de la saisie textarea de l'admin.
PDF download	Blob download via programmatic <a> click. 202=retry, 4xx=unavailable, 200=save dialog. Filename: rapport-{reference}.pdf. Téléchargement blob via clic <a> programmatique. 202=réessayer, 4xx=non disponible, 200=boîte enregistrer.

getCurrentUser	Reads cyberaudit:session from localStorage directly (not useAuth hook). try/catch handles JSON parse errors. Lit cyberaudit:session depuis localStorage directement. try/catch gère les erreurs de parse JSON.
5 finding fields	severity (select: Critical/High/Medium/Low), asset (required, maxLength 200), description, recommendation, plus auto-index key. severity (select), asset (requis, maxLength 200), description, recommendation, plus clé index automatique.

Full flow: Admin navigates to /admin/report/:reference → Promise.all loads report + audit → setEditMode(isAdmin && status!='completed') → if editMode: admin fills findings/summary/verdict, previewScore updates live → clicks Generate → submitting=true → modal shows cycling messages → POST to backend → Django computes score/grade, renders Jinja2 HTML, WeasyPrint creates PDF → auto-completes audit → reload → setEditMode(false) → read mode shows grade badge + score bar + findings + Download PDF button.

Flux complet : Admin navigue vers /admin/report/:reference → Promise.all charge rapport + audit → setEditMode(isAdmin && status!='completed') → si editMode : admin remplit findings/summary/verdict, previewScore se met à jour live → clique Generate → submitting=true → modal affiche messages cycliques → POST backend → Django calcule score/note, rend HTML Jinja2, WeasyPrint crée PDF → marque audit complète → rechargement → setEditMode(false) → mode lecture affiche badge note + barre score + findings + bouton Télécharger PDF.